

Лабораторная работа №6

Решение моделей в непрерывном и дискретном времени

Чемоданова Ангелина Александровна

22 сентября 2025

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Чемоданова Ангелина Александровна
- Студентка НФИбд-02-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132226443@pfur.ru
- <https://github.com/aachemodanova>



Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

Вспомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u .

Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет `differentialEquations.jl`.

Модель экспоненциального роста

```
import Pkg
using DifferentialEquations
using Plots
```

Julia

```
# задаём описание модели с начальными условиями:
```

```
a = 0.98
```

```
f(u,p,t) = a*u
```

```
u0 = 1.0
```

```
# задаём интервал времени:
```

```
tspan = (0.0,1.0)
```

```
# решение:
```

```
prob = ODEProblem(f,u0,tspan)
```

```
sol = solve(prob)
```

Julia

```
retcode: Success
```

```
Interpolation: 3rd order Hermite
```

```
t: 5-element Vector{Float64}:
```

```
0.0
```

```
0.10042494449239292
```

```
0.3521860297865888
```

```
0.6934436122197829
```

```
1.0
```

```
u: 5-element Vector{Float64}:
```

```
1.0
```

```
1.1034222047865465
```

```
1.4121908713484919
```

```
1.9730384457359198
```

```
2.6644561424814266
```

Рис. 1: График модели экспоненциального роста

Модель экспоненциального роста

строим графики:

```
plot(sol, linewidth=5, title="Модель экспоненциального роста", хaxis="Время", уaxis="u(t)", label="u(t)")  
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

Julia

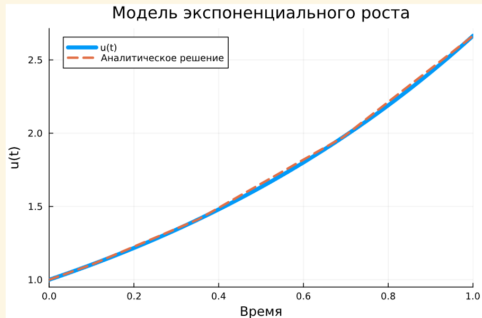


Рис. 2: График модели экспоненциального роста

Модель экспоненциального роста

```
sol = solve(prob, abstol=1e-8, reltol=1e-8)
println(sol)
# строим график:
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", хaxis="Время", уaxis="u(t)", label="Численное решение")
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое решение")
```

Julia

ODESolution{Float64, 1, Vector{Float64}, Nothing, Nothing, Vector{Float64}, Vector{Vector{Float64}}, Nothing, ODEProblem{Float64, Tuple{Float64, Float64}, false, SciMLBase.

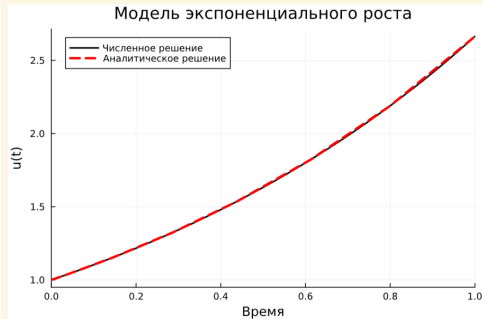


Рис. 3: График модели экспоненциального роста (задана точность решения)

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка.

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующем усечением до первых-вторых гармоник.

Решение системы неустойчиво на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления.

Система Лоренца

```
# задаём описание модели:
function lorentz!(du,u,p,t)
    β,ρ,β = p
    du[1] = β*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end

# задаём начальное условие:
u0 = [1.0,0.0,0.0]

# задаём значения параметров:
p = (10,28,8/3)

# задаём интервал времени:
tspan = (0.0,100.0)

# решение:
prob = ODEProblem(lorentz!,u0,tspan,p)
sol = solve(prob)

# строим график:
plot(sol, vars=(1,2,3), lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=false)
```

Julia

Аттрактор Лоренца

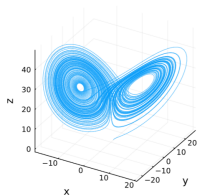


Рис. 4: Аттрактор Лоренца

отключаем интерполяцию:

```
plot(sol,vars=(1,2,3),denseplot=false, lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=false)
```

Julia

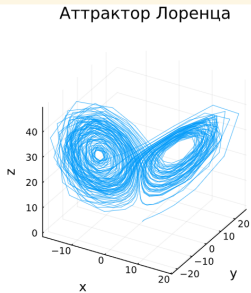


Рис. 5: Аттрактор Лоренца (интерполяция отключена)

Модель Лотки–Вольтерры

```
function lotka_voltterra(du, u, p, t)
    x, y = u
    a, b, c, d = p
    du[1] = a * x - b * x * y
    du[2] = -c * y + d * x * y
end
u0 = [1.0, 1.0]
p = [1.5, 1.0, 3.0, 1.0]
tspan = (0.0, 10.0)
prob = ODEProblem(lotka_voltterra, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, label=["Жерты" "Хищники"], color="black",
    linestyle = [:solid :dash], title="Модель Лотки-Вольтерры",
    xlabel="Время", ylabel="Размер популяции")
```

Julia

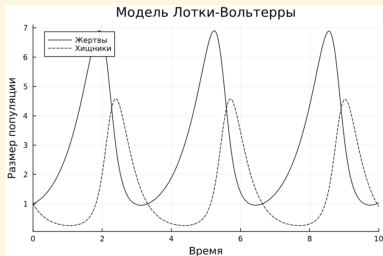


Рис. 6: Модель Лотки–Вольтерры: динамика изменения численности популяций

Модель Лотки–Вольтерры

```
# фазовый портрет:  
plot(sol,vars=(1,2), color="black", xaxis="Жертвы",yaxis="Хищники", legend=false)
```

Julia

```
Warning: To maintain consistency with solution indexing, keyword argument vars will be removed in a future version. Please use keyword argument idxs instead.  
[ caller = ip:0x0  
[ @ Core :-1
```

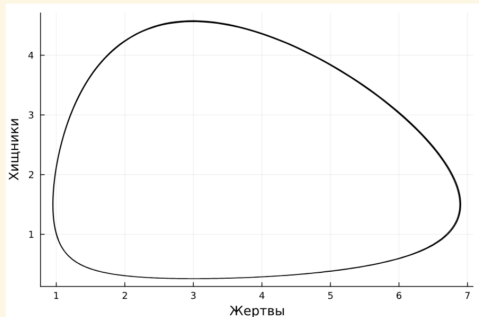


Рис. 7: Модель Лотки–Вольтерры: фазовый портрет

Самостоятельное выполнение

1) Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
f(u, p, t) = a * u
b = 0.5
c = 0.1
a = b - c
u0 = 1.0
tspan = (0.0, 10.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Модель Мальтуса", label="Численное решение", color="pink", linewidth = 3)
```

Julia

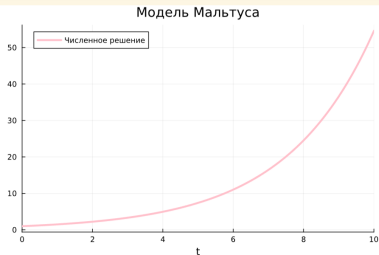


Рис. 8: Решение задания №1

Самостоятельное выполнение

```
anim = @animate for i in 1:length(sol.t)
|   plot(sol.t[1:i], sol.u[1:i], linewidth=3, title="Модель Мальтуса", xlabel="Время", ylabel="Численность популяции", legend=false, color="pink")
end
gif(anim, "malthus_population_growth.gif", fps=15)
```

Julia

Info: Saved animation to c:\Users\angelina\Downloads\malthus_population_growth.gif
@ Plots <C:\Users\angelina\.julia\packages\Plots\Ec1L1\src\animation.jl:156>



Рис. 9: Решение задания №1

Самостоятельное выполнение

2) Реализовать и проанализировать логистическую модель роста популяции. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
f(u, p, t) = r * u * (1 - u/k)
r = 0.5
k = 100
u0 = 1.0
tspan = (0.0, 20.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Логистическая модель роста популяции", label="Численность популяции", color="pink", linewidth = 3)
```

Julia

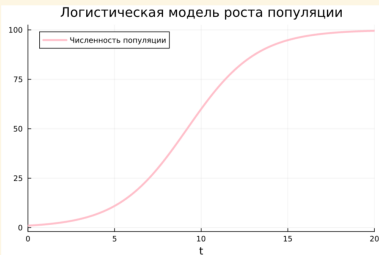


Рис. 10: Решение задания №2

Самостоятельное выполнение

```
anim = @animate for t in 0:0.5:20
|   plot(sol.t[sol.t .<= t], sol.u[sol.t .<= t], label="Численное решение", xlim=(0, 20), ylim=(0, k), lw=2, color="pink", title="Логистическая модель роста", xlabel="Время", ylabel="Численность популяции")
end
gif(anim, "logistic_growth.gif", fps=10)
```

Julia

Info: Saved animation to c:\Users\angelina\Downloads\logistic_growth.gif
@ Plots <C:\Users\angelina\.julia\packages\Plots\Ec111\src\animation.jl:156>

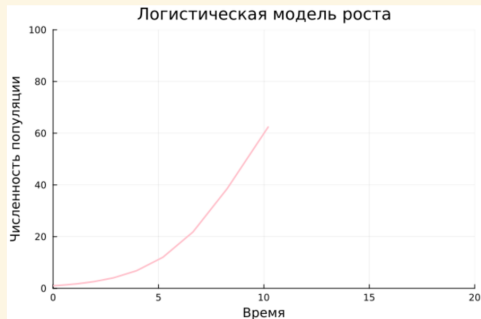


Рис. 11: Решение задания №2

3) Реализовать и проанализировать модель эпидемии Кермака–Маккендрика (SIR-модель). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
function sir(u, p, t)
    β, γ = p
    s, i, r = u
    ds = -β * s * i      # Восприимчивые
    di = β * s * i - γ * i # Инфицированные
    dr = γ * i           # Выздоровевшие
    return [ds, di, dr]
end
```

Julia

sir (generic function with 1 method)

Рис. 12: Решение задания №3

Самостоятельное выполнение

```
u0 = [0.99, 0.01, 0.0] # 99% восприимчивых, 1% инфицированных
β = 0.3 # Коэффициент передачи инфекции
γ = 0.1 # Коэффициент выздоровления
p = (β, γ)
tspan = (0.0, 100.0)
prob_sir = ODEProblem(sir, u0, tspan, p)
sol_sir = solve(prob_sir, Tsit5())
plot(sol_sir, title="SIR model", label=["S" "I" "R"])
```

Julia

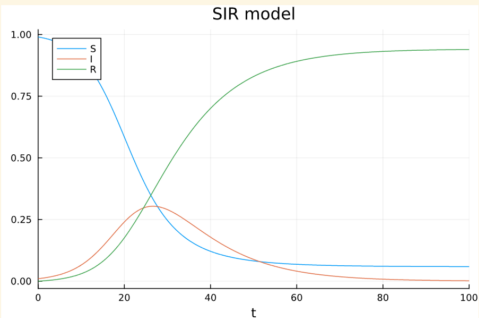


Рис. 13: Решение задания №3

Самостоятельное выполнение

```
anim = @animate for t in 1:length(sol_sir.t)
    plot(sol_sir.t[1:t], [u[1] for u in sol_sir.u[1:t]], label="Восприимчивые", color=:blue, linewidth=2)
    plot!(sol_sir.t[1:t], [u[2] for u in sol_sir.u[1:t]], label="Инфицированные", color=:red, linewidth=2)
    plot!(sol_sir.t[1:t], [u[3] for u in sol_sir.u[1:t]], label="Выздоровевшие", color=:green, linewidth=2)
    title!("Модель эпидемии SIR")
    xlabel!("Время")
    ylabel!("Численность")
end
gif(anim, "sir_model_animation.gif", fps=120)
```

Julia

Info: Saved animation to c:\Users\angelina\Downloads\sir_model_animation.gif
@ Plots <c:\Users\angelina\julia\packages\Plots\Ecili\src\animation.jl:156>

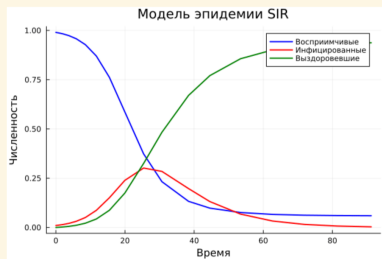


Рис. 14: Решение задания №3

4) Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed). Исследуйте, сравните с SIR.

```
function seir(u, p, t)
    (s, e, i, r) = u
    (β, γ, δ) = p
    N = s+e+i+r
    ds = -(β * s * i) / N      # Восприимчивые
    de = (β * s * i) / N - δ * e # Экспонированные
    di = δ * e - γ * i         # Инфицированные
    dr = γ * i                 # Выздоровевшие
    return [ds, de, di, dr]
end
```

Julia

seir (generic function with 1 method)

Рис. 15: Решение задания №4

```
dt_ = 0.1
u0 = [980.0, 10.0, 10.0, 0.0]
p = [0.3, 0.1, 0.2]
tmax = 60.0
tspan = (0.0, tmax)
prob_1 = ODEProblem(seir, u0, tspan, p)
sol = solve(prob_1, dt = dt_)
plot(sol, title="SEIR-model", label=["S" "E" "I" "R"])
```

Julia

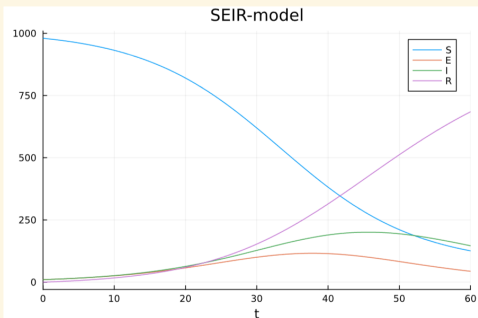


Рис. 16: Решение задания №4

5) Для дискретной модели Лотки–Вольтерры найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

```
a = 2
c = 1
d = 5
X0 = [0.5, 0.3]
tspan = 50
X1 = zeros(tspan)
X2 = zeros(tspan)
X1[1] = X0[1]
X2[1] = X0[2]
for t in 1:tspan-1
    X1[t+1] = a * X1[t] * (1-X1[t]) - X1[t] * X2[t]
    X2[t+1] = -c * X2[t] + d * X1[t] * X2[t]
end
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")
plot(1:tspan, X2, label="Хищники(X2)")
```

Julia

Рис. 17: Решение задания №5

```
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")  
plot!(1:tspan, X2, label="Хищники(X2)")
```

Julia

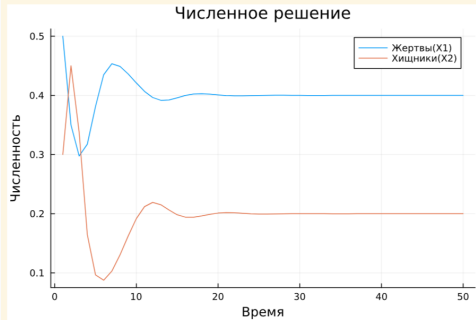


Рис. 18: Решение задания №5

```
plot(X1, X2, xlabel="Жертвы(X1)", ylabel="Хищники(X2)", title="Фазовый портрет")
```

Julia

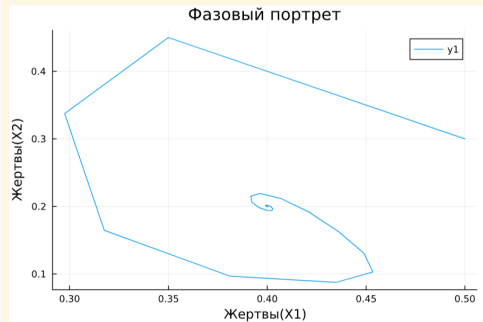


Рис. 19: Решение задания №5

6) Реализовать на языке Julia модель отбора на основе конкурентных отношений. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
function competition(du, u, p, t)
    α, β = p
    x, y = u
    du[1] = α * x - β * x * y
    du[2] = α * y - β * x * y
end
```

Julia

competition (generic function with 1 method)

Рис. 20: Решение задания №6

```
tspan = (0.0, 50.0)
u0 = [10.0, 5.0]
p = [1, 0.01]
prob = ODEProblem(competition, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, label=["Популяция x" "Популяция y"], title="Модель конкуренции", xlabel="Время", ylabel="Популяция", linewidth=2)
```

Julia

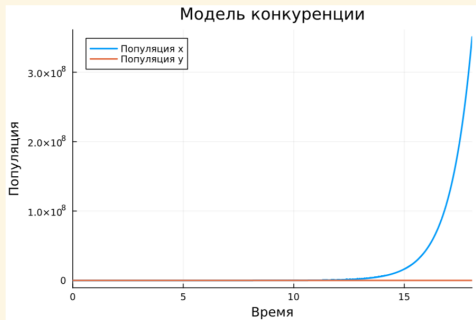


Рис. 21: Решение задания №6

```
plot(sol, idxs=(1, 2), title="Конкурентные отношения")
```

Julia

Конкурентные отношения

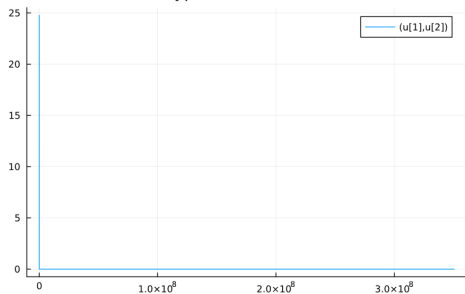


Рис. 22: Решение задания №6

7) Реализовать на языке Julia модель консервативного гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
function harmonic(u, p, t)
    x, y = u
    Y, omega = p
    dx = y
    dy = -2*Y*y - omega^2 * x
    return [dx, dy]
end
```

✓ 0.0s

Julia

harmonic (generic function with 1 method)

Рис. 23: Решение задания №7

Самостоятельное выполнение

```
tspan = (0.0, 2*pi)
u0 = [-0.5, 0]
p = [0, 3.5]
prob = ODEProblem(harmonic, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, title="Гармонический осциллятор")
```

✓ 0.1s

Julia

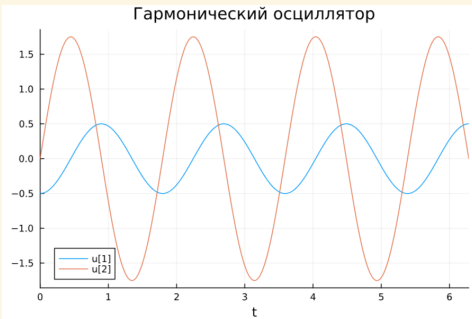


Рис. 24: Решение задания №7

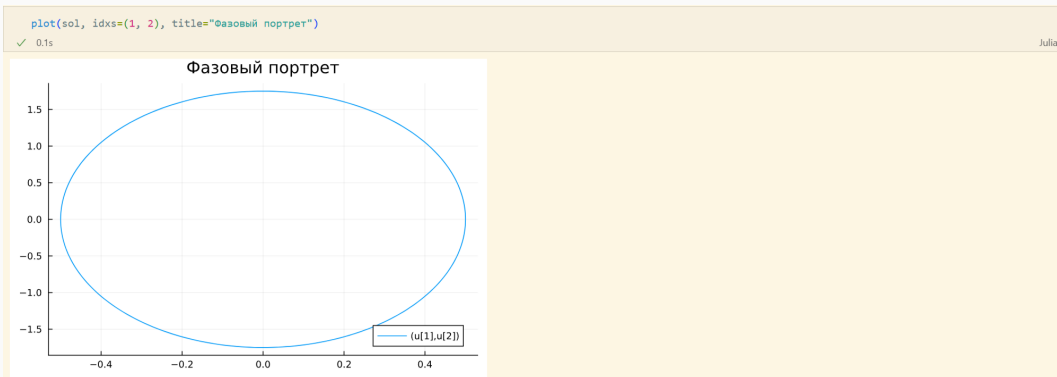


Рис. 25: Решение задания №7

Самостоятельное выполнение

8) Реализовать на языке Julia модель свободных колебаний гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

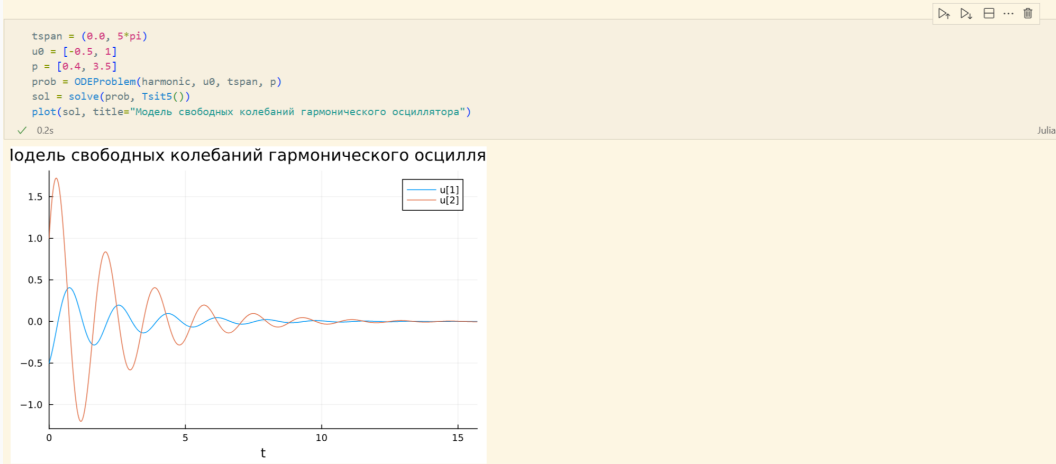


Рис. 26: Решение задания №8

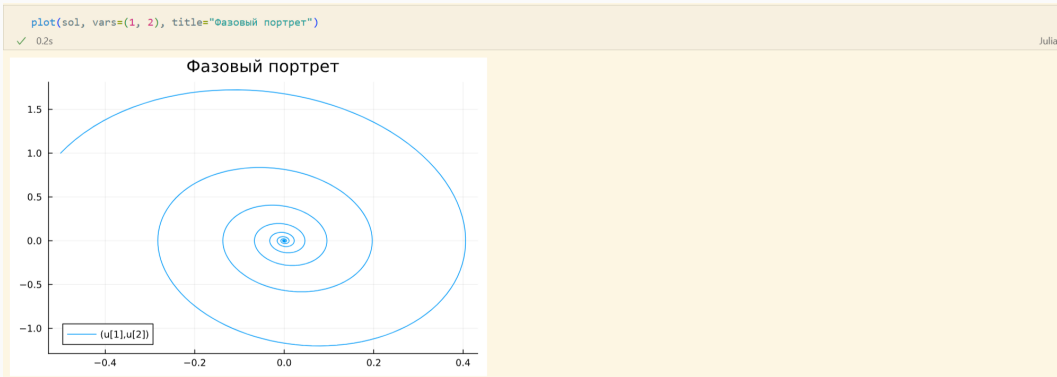


Рис. 27: Решение задания №8

В результате выполнения данной лабораторной работы мы освоили специализированные пакеты для решения задач в непрерывном и дискретном времени.